

Från Pythonkod till interaktiv träningsapp

Hitta din träningsform | Projektrapport

Syfte

Jag valde att fördjupa mig i Streamlit eftersom jag har funderat på hur man kan göra Pythonkod och data användbara för personer som inte själva kan programmera.

Mitt syfte var att lära mig hur Streamlit fungerar och hur man kan använda det för att bygga ett interaktivt gränssnitt med Python.

För att testa detta byggde jag en träningsapp där användaren får träningsförslag utifrån sina önskemål.

Jag ville också undersöka hur appen kan samla in användarens svar och använda dem för att visa statistik.

Området och dess relevans

Streamlit är ett Pythonbibliotek som gör det möjligt att bygga interaktiva appar utan att behöva kunna traditionell webbutveckling.

Med Streamlit kan man skapa ett gränssnitt där användaren till exempel fyller i ett formulär, klickar på en knapp och får ett resultat direkt på skärmen.

Som Data Scientist handlar arbetet mycket om att samla in, bearbeta och analysera data. Men för att resultatet ska komma till nytta behöver det också kunna presenteras på ett sätt som andra förstår och kan använda.

Med Streamlit kan en Data Scientist exempelvis göra en analys interaktiv, så att en användare själv kan välja vad den vill undersöka och se hur resultatet förändras.

Viktiga begrepp

Hur Streamlit kör koden

I Streamlit skriver man appens gränssnitt och funktioner med Pythonkod. Användaren kan sedan göra val i exempelvis formulär och klicka på knappar för att få ett resultat.

Streamlit kör koden på nytt, uppifrån och ner, så fort användaren gör något i appen.

Det gör appen interaktiv, men kan också skapa problem. Jag stötte på två sådana problem i mitt projekt.

Cache – slippa läsa in samma data om och om igen

Eftersom koden körs om kan appen läsa in samma CSV-fil varje gång användaren klickar på en knapp, även om filen inte har ändrats. Det är onödigt arbete, särskilt om man använder större datamängder.

Med `@st.cache_data` kan Streamlit spara resultatet av en funktion, till exempel inläsningen av en CSV-fil, och återanvända det nästa gång funktionen anropas. Då behöver appen inte läsa in samma data på nytt vid varje knapptryckning.

Session state – behålla information mellan knapptryckningar

När Streamlit kör om koden kan information som bara visas efter ett visst knapptryck försvinna från skärmen.

Det här blev tydligt när jag lade till en knapp för att spara användarens inställningar. När jag klickade på den försvann träningsförslagen från skärmen, eftersom koden kördes om och knappen för att hitta träningsförslag inte längre var aktiv.

Genom att spara förslagen i `st.session_state` kunde jag se till att de låg kvar även efter att användaren klickat på Spara.

Genomförande

Jag började med att skapa ett mindre dataset med tio olika träningsformer i en CSV-fil. För varje träningsform angav jag bland annat hur väl den passar olika träningsmål, hur lång tid ett pass tar, intensitet, socialt upplägg och träningsmiljö. Jag valde att använda ett litet, manuellt sammanställt dataset för att kunna fokusera på att lära mig Streamlit och hur appens olika delar fungerar tillsammans.

Därefter byggde jag gränssnittet i Streamlit. Användaren börjar med att skriva sitt namn och får sedan svara på frågor om sina träningsmål och önskemål. Jag använde ett formulär så att svaren skickas vidare tillsammans när användaren klickar på *”Hitta min träningsform”*.

För att kunna ge träningsförslag skapade jag en enkel, regelbaserad matchningsmodell i Python. Den jämför användarens svar med träningsformernas egenskaper och räknar ut en poäng för varje träningsform. Jag valde att låta träningsmål och intensitet påverka poängen mer än exempelvis socialt upplägg och miljö. Om användaren svarar Spelar ingen roll på någon av de två sista frågorna tas den faktorn inte med i beräkningen. Appen visar sedan de tre träningsformer som fått högst poäng och ett diagram över de fem bästa matchningarna.

Jag ville också undersöka hur appen kan samla in ny data. Därför lade jag till en knapp som sparar användarens svar och bästa matchning i en CSV-fil. Om samma namn redan finns i filen uppdateras den befintliga raden. På en separat statistiksida använder jag Pandas för att sammanställa hur många användare som fått respektive träningsform som bästa matchning. Resultatet visas både i en tabell och i ett diagram som jag har skapat med Altair.

Hitta din träningsform

Få personliga träningsförslag utifrån dina mål och preferenser. Svara på några korta frågor så matchar appen dig med träningsformer som kan passa dig.

Appen använder en förenklad matchningsmodell baserad på bland annat träningsmål, tid, intensitet, socialt upplägg och miljö.

Hej Shara!

Berätta lite om hur du vill träna

Svara på några korta frågor så hittar vi träningsformer som matchar dina preferenser.

Vad är ditt främsta mål med träningen?

Bli starkare

Hur mycket tid vill du lägga på ett träningspass?

30 minuter eller mindre

Vilken intensitet föredrar du?

Låg

Hur vill du helst träna?

☒ Själv
☐ Spelar ingen roll
☐ Tillsammans med andra

Var tränar du helst?

☒ Inomhus
☐ Spelar ingen roll
☐ Utomhus

Hitta min träningsform

Appens formulär för träningsmål och önskemål.

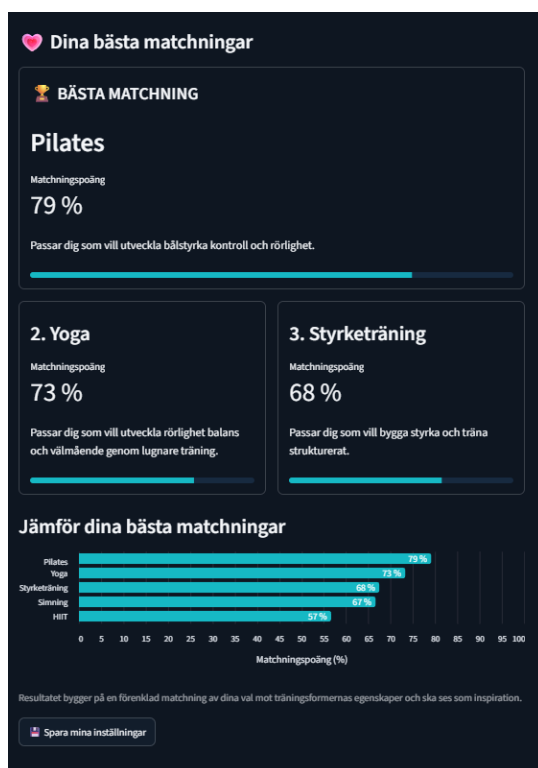
Till sist publicerade jag appen via Streamlit Community Cloud så att den går att testa direkt i webbläsaren.

Resultat

Resultatet av arbetet blev en fungerande träningsapp som går att använda direkt i webbläsaren. Med hjälp av ett enkelt gränssnitt kan användaren göra egna val och ta del av resultat från Pythonkoden, utan att själv behöva kunna programmera.

Appen presenterar den bästa träningsmatchningen på ett tydligt sätt och låter användaren jämföra olika alternativ i ett diagram. Med en knapptryckning kan användaren också spara sina svar, som sedan ligger till grund för statistiken i appen.

För mig visar resultatet hur Streamlit kan göra kod och data till något som andra faktiskt kan använda och förstå.



Träningsförslag och jämförelse av matchningspoäng.

Begränsningar och möjliga förbättringar

Appen är en mindre prototyp som bygger på tio träningsformer och data som jag själv har sammanställt. Matchningsmodellen är också förenklad: jag har valt vilka faktorer som ska ingå och hur mycket de ska påverka resultatet. En möjlig vidareutveckling skulle vara att använda ett större dataset med verklig data och undersöka om en maskininlärningsmodell kan ge mer träffsäkra rekommendationer.

När jag publicerade appen via Streamlit Community Cloud blev det också tydligt att den lokala CSV-filen har begränsningar som lösning för att spara användarnas svar över tid. Nya svar kan försvinna när appen startas om. Om flera personer använder appen samtidigt kan det dessutom uppstå problem när deras svar ska skrivas till samma fil.

Om jag utvecklade appen vidare skulle jag därför använda en extern databas. Då skulle användarnas svar kunna sparas mer långsiktigt och hanteras på ett bättre sätt även när flera använder appen.

Koppling till yrkesrollen

Som Data Scientist räcker det inte alltid att kunna analysera data och ta fram resultat. Det är också viktigt att kunna presentera resultaten så att andra kan förstå och använda dem, även om de inte själva kan programmera.

Streamlit kan vara ett användbart verktyg för att göra just det. Man kan till exempel bygga en app där kollegor eller kunder själva kan utforska en analys, ändra olika värden och direkt se hur resultatet påverkas.

Källor

Streamlit – officiell dokumentation

- [Get started with Streamlit](#) – introduktion till Streamlit och hur man bygger appar med Python.
- [Caching and state](#) – hur Streamlit hanterar omkörning av koden, cache och information som behöver finnas kvar mellan användarens val.
- [Session State](#) – hur `st.session_state` används för att behålla information när appen körs om.
- [st.cache_data](#) – hur resultat från exempelvis datainläsning kan sparas och återanvändas.
- [Deploy your app on Community Cloud](#) – hur en Streamlit-app publiceras och delas via webbläsaren.

Altair – officiell dokumentation

- [Altair Documentation](#) – dokumentation för biblioteket som används för att skapa appens diagram.

Självreflektion

Det jag framför allt har lärt mig är hur man kan använda Streamlit för att göra Pythonkod och data till en interaktiv app. Innan jag började hade jag inte byggt något liknande, och jag visste inte hur enkelt det kunde vara att skapa ett gränssnitt där andra kan göra egna val och se resultatet direkt. Jag har också fått en bättre förståelse för hur Streamlit kör koden och varför man ibland behöver använda `cache` och `st.session_state`.

Det svåraste var att förstå vad som händer när Streamlit kör om koden. När jag lade till knappen för att spara användarens inställningar försvann träningsförslagen, trots att de nyss hade visats. Det tog ett tag innan jag förstod varför det hände och hur jag kunde lösa det. När jag väl fick det att fungera kände jag att jag hade förstått något viktigt om hur Streamlit fungerar.

Det tekniska val jag är mest nöjd med är att jag delade upp koden i olika filer för gränssnitt, datahantering, matchning och statistik. Det gjorde det lättare för mig att hålla ordning på projektet och förstå vad varje del ansvarar för. Jag är också nöjd med att appen inte bara visar ett resultat, utan att användaren kan spara sina svar och se hur de bidrar till statistiken.

Om jag började om idag skulle jag faktiskt göra det mesta på samma sätt. Jag är nöjd med hur jag har arbetat metodiskt och byggt upp appen steg för steg. Det har gjort det lättare att förstå de olika delarna och hur de hänger ihop. Däremot hade arbetet förmodligen gått lite snabbare nu när jag vet mer om hur Streamlit fungerar. Framför allt hade jag nog sluppit lägga lika mycket tid på att fundera över varför träningsförslagen försvann när jag klickade på Spara!

Om jag fortsatte utveckla appen skulle nästa steg vara att koppla den till en extern databas så att användarnas svar kan sparas långsiktigt. På sikt skulle det också vara intressant att arbeta med ett större dataset och undersöka möjligheten att använda maskininlärning för rekommendationerna.

Min egen bedömning är att arbetet motsvarar VG. Jag har byggt en fungerande app, men framför allt har jag fördjupat mig i hur Streamlit fungerar och använt den förståelsen för att lösa problem som uppstod under arbetet. Jag kan förklara mina tekniska val, hur appens olika delar samverkar och vilka begränsningar lösningen har.